Loan Eligibility Checker

Example program: validation, tiered decisions and f-string formatting

The problem

Decide whether an applicant qualifies. Hard rules reject immediately; the credit score then picks the outcome. (Illustrative rules only.)

Check	Rule
Age	`21 <= age <= 60`
Income	monthly income `>= 25000`
Debt ratio	existing EMI / income `<= 0.50`
Amount	loan `<= 20 x` monthly income
Score 750+	approved at 9.5%
Score 650-749	approved at 11.5%
Score 600-649	manual review
Score below 600	rejected

Try it

```
print(check_loan(30, 60000, 780, 10000, 800000))
print(check_loan(35, 40000, 700, 5000, 500000))
print(check_loan(28, 30000, 620, 0, 300000))
print(check_loan(19, 50000, 800, 0, 100000))
print(check_loan(40, 30000, 700, 20000, 100000))
print(check_loan(30, 30000, 800, 0, 900000))
```

OUTPUT

```
Approved at 9.5%
Approved at 11.5%
Manual review
Rejected: age must be 21-60
Rejected: DTI 67% > 50%
Rejected: max loan 600,000
```

Decision flow

```
age 21-60? ──────────────no─► Rejected
    | yes
income >= 25000? ────────no─► Rejected
    | yes
EMI / income <= 50%? ────no─► Rejected
    | yes
amount <= 20 x income? ──no─► Rejected
    | yes
score 750+  ─► Approved 9.5%
score 650+  ─► Approved 11.5%
score 600+  ─► Manual review
otherwise   ─► Rejected
```

Boundary cases to test

Most bugs hide on the edges: check each limit exactly, one below and one above.

Input	Expected
`age` 21 or 60	accepted (`<=` is inclusive)
`age` 20 or 61	rejected
`income` exactly 25000	accepted (`<` is strict)
DTI exactly 0.50	accepted (`>` is strict)
`score` 750 / 650 / 600	lands in the better tier

f-string number formats

```
f"{0.1667:.0%}"          # → '17%'
f"{800000:,}"            # → '800,000'
f"{9.5:.1f}%"            # → '9.5%'
f"{3.14159:.2f}"         # → '3.14'
f"{42:>6}"               # → '    42'
```

The program

```
def check_loan(age, income, score, emi, amount):
    if not 21 <= age <= 60:
        return "Rejected: age must be 21-60"
    if income < 25000:
        return "Rejected: income below 25,000"
    dti = emi / income            # debt-to-income
    if dti > 0.5:
        return f"Rejected: DTI {dti:.0%} > 50%"
    if amount > 20 * income:
        return f"Rejected: max loan {20 * income:,}"
    if score >= 750:
        return "Approved at 9.5%"
    elif score >= 650:
        return "Approved at 11.5%"
    elif score >= 600:
        return "Manual review"
    return "Rejected: score below 600"
```

Variation: collect every reason

Instead of stopping at the first failure, gather all of them in a list.

```
def reasons(age, income, score):
    why = []
    if not 21 <= age <= 60:
        why.append("age")
    if income < 25000:
        why.append("income")
    if score < 600:
        why.append("credit score")
    return why
print(reasons(19, 20000, 580))
print(reasons(30, 50000, 700))
```

OUTPUT

```
['age', 'income', 'credit score']
[]
```

Chained ternary for tiers

A compact alternative to `if/elif` when each branch is just a value.

```
def tier(score):
    return ("A" if score >= 750 else
            "B" if score >= 650 else
            "C" if score >= 600 else "D")
print(tier(780), tier(700), tier(620), tier(500))
print(tier(750), tier(649))
```

OUTPUT

```
A B C D
A C
```

Validate input before using it

```
age = input("Age: ")
if not age.isdigit():
    print("Digits only, please")
elif int(age) < 21:
    print("Too young for this loan")
else:
    print("Age accepted")
```

OUTPUT

```
Age accepted
```

TIP `isdigit()` rejects "abc", "-5" and "2.5" before `int()` can crash.

REMEMBER

- Validate input first, then apply hard rules, then the nuanced tiers.
- Early `return` keeps each rule flat and its failure message right beside it.
- Chained comparison `21 <= age <= 60` reads exactly like the business rule.
- In an `if/elif` chain, test the highest threshold first (`>= 750` before `>= 650`).
- Keep thresholds in named constants (`MIN_INCOME = 25000`) so policy changes are one-line edits.
- This is a learning model, not financial advice: real lenders weigh many more factors.